

LLM-as-a-Judge 평가 방법론



정확성 vs 근거성

정확성은 실제 사실과 비교하고, 근거성은 제공된 컨텍스트와 비교한다.

정확성 (Accuracy)

- 실제 사실과 맞는 답인가를 본다
- 예: "재구매 고객 LTV는 4.8배", 사실이면 통과
- 정답지가 있어야 채점할 수 있다

채점에 정답지 필요

근거성 (Faithfulness)

- 제공된 컨텍스트에 충실한 답인가를 본다
- 예: 문서에 없는 4.8배를 말하면 실패
- 정답지 없이도 채점할 수 있다

정답지 없이 채점 가능

정답지 없이도 채점할 수 있는 근거성이 RAG 평가의 기본 기준이 된다.



분류 태스크와 생성 태스크의 차이

분류에는 정해진 답이 있다. 생성은 표현이 달라도 정답이고, 표현이 같아도 오답일 수 있다.

분류 태스크

"이 메일은 스팸인가?"
정답 라벨은 '스팸' 하나

채점 = 라벨 일치 여부
정확도를 즉시 계산

생성 태스크 · 근거: "라이브커머스 이용 경험률은 30대가 66.2%로 가장 높다"

모두 정답 · 표현은 제각각
"30대 이용률이 66.2%로 1위이다"

"가장 많이 이용한 세대는
30대(66.2%)입니다"

"30대가 제일 많이 봐요
(66.2%)"

모두 오답 · 표현은 거의 그대로
"라이브커머스 이용 경험률은
40대가 66.2%로 가장 높다"
연령대가 바뀜 ×

"30대가 66.2%로
가장 낮다"
한 단어 반전 ×

왼쪽은 단어가 모두 다르고, 오른쪽은 단어가 거의 같다. 정답 여부는 그 반대이다.

겹침 기반 지표의 역전 현상

n-gram 겹침으로 측정하면 오답("40대가 66.2%로 가장 높다")이 정답 표현들보다 높은 점수를 받는다.

따라서 의미와 근거를 읽는 채점이 필요하다.



생성 평가의 난점: 전통 지표의 한계

생성에는 유일한 정답이 없다. 겹침을 세는 지표로는 의미를 채점할 수 없다.

정확 일치 (Exact Match)
"4.8배" ≠ "신규 대비 4.8배 높다"
같은 뜻, 다른 표현이면 0점

×

BLEU · ROUGE
(n-gram 겹침)

표현이 다르면 의미가 같아도 낮고,
표현이 겹치면 틀려도 높다

×

임베딩 유사도

의미는 비교하지만
근거 충실성은 확인하지 못한다

×

생성 평가의 본질적 난점

유효한 답이 여럿 · 부분 정답의 정도 문제 · 과잉 정보와 누락의 비대칭



의미를 읽는 채점자
LLM-as-a-Judge
생성 채점의 도구

겹침 기반 지표는 근거 기반 QA 채점에 맞지 않는다. 따라서 루브릭을 갖춘 LLM에 채점을 맡긴다.

reference-based

- 사람이 만든 정답(ground truth)과 비교해 채점한다.
- Context Recall처럼 정답 문서·정답 답변이 필요하다.

reference-free

- 정답 없이 질문·컨텍스트·답 사이의 정합성으로 채점한다.
- Faithfulness처럼 새 질문에도 바로 적용할 수 있다.

정답이 없어도 평가는 가능하다.
대신 무엇을 기준으로 채점하는지가 지표마다 다르다.



LLM-as-a-Judge의 개념

다른 LLM의 출력을 채점하는 LLM이다. 사람 채점의 속도·비용 한계를 보완하는 자동 평가자이다.

기존 평가의 한계

규칙 기반 분류

분류기는 "정상 또는 생성 실패"까지만 판별했다.

사람 채점

정확하지만 시간과 비용이 많이 들고,
채점자마다 기준이 달라진다.



LLM-as-a-Judge 도입

장점 · 속도와 규모

문항 수천 개도 사람과 비교할 수 없는
속도로 처리할 수 있다.

단점 · 편향과 불안정성

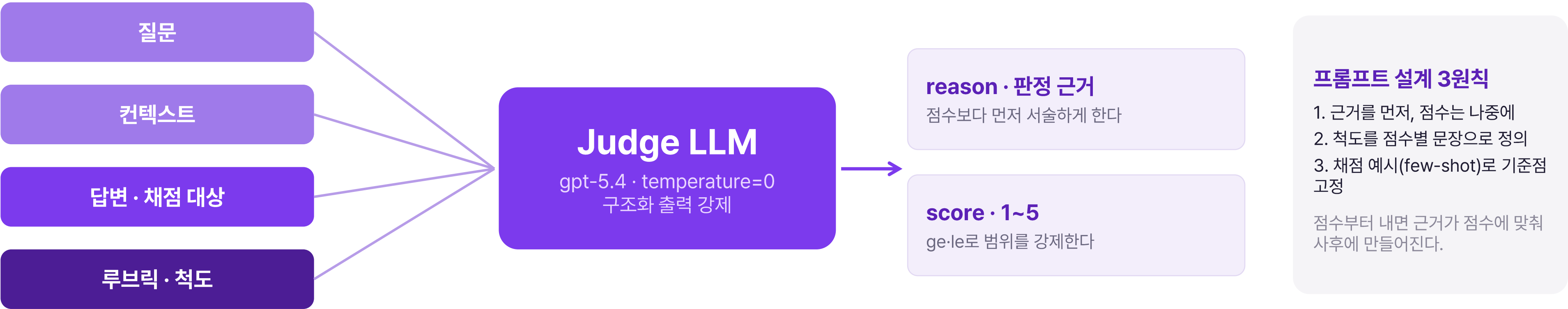
Judge도 LLM이라 편향과 불안정성이 있다.

Judge를 신뢰하기 위한 조건까지 이어서 다룬다.



Judge 평가의 구조

루브릭을 갖춘 LLM이 채점자가 된다. 단, 아래 네 편향을 줄여야 결과를 신뢰할 수 있다.



네 가지 편향과 완화 장치

위치 편향 먼저 나온 답을 선호 ▼ 순서 교체 이중 판정	장황함 편향 길어 보이면 후한 점수 ▼ 근거 중심 루브릭	자기선호 편향 같은 모델의 답을 선호 ▼ 다른 모델·교차 채점	점수 불안정 실행마다 점수 변동 ▼ 반복 채점·표준편차
--	--	---	---

입력 네 가지와 출력 두 가지를 고정하면 같은 채점을 재현할 수 있다. 편향 완화 장치는 그 위에 더한다.



루브릭 설계 원칙

채점 품질은 모델보다 루브릭의 영향을 크게 받는다. 다음 6가지를 갖추면 점수 해석의 변동 폭을 줄일 수 있다.

점수별 조건 문장화

5점·3점·1점이 각각 어떤 상태인지 문장으로 적는다. 등급별 설명을 둔 루브릭이 사람 판정과 더 잘 맞는다는 보고가 있다.

근거 먼저, 점수 나중

판정 근거를 먼저 서술하게 하고 점수를 마지막에 받는다. 점수부터 내면 근거를 뒤에 맞추게 된다.

기준 하나에 척도 하나

관련성과 완전성처럼 성격이 다른 기준을 한 점수에 섞지 않는다. 섞으면 어느 쪽이 낮은지 알 수 없다.

모호어 배제

"적절하다", "충분하다" 같은 표현은 판정마다 달라진다. 관찰할 수 있는 조건으로 바꿔 쓴다.

예시로 기준점 고정

등급별 예시 답변을 함께 준다. 예시 자체가 척도를 이동시키므로 버전으로 관리한다.

구조화 출력

근거와 점수를 정해진 형식으로 받아 범위 이탈과 파싱 실패를 막는다.

루브릭은 코드와 같이 버전으로 관리한다. 문장이나 예시를 바꾸면 이전 점수와 직접 비교하기 어렵다.



Judge 평가 기준 4종

기준마다 별도의 루브릭을 두고 채점한다. 4개 기준은 답변의 서로 다른 측면을 다룬다.

1 Relevance 관련성 질문에 대한 답인가 채점 포인트 동문서답이면 0점부터	2 Coherence 일관성 논리가 이어지는가 채점 포인트 앞뒤 모순·비약 검사	3 Faithfulness 충실성 컨텍스트에 충실한가 채점 포인트 RAG 평가의 핵심 기준	4 Completeness 완전성 빠뜨린 근거가 없는가 채점 포인트 부분 답변 감점
--	---	---	---

네 기준 모두 루브릭을 설계하고, 실습은 대표 기준인 Faithfulness로 구현한다.



관련성·완전성 루브릭 설계

관련성만 높이면 답이 짧아지고, 완전성만 높이면 군더더기가 늘어난다. 두 기준을 함께 보는 이유이다.

기준	5점	3점	1점
관련성 (Relevance)	질문이 요구한 내용에만 답한다	답은 있으나 불필요한 내용이 섞인다	질문과 다른 내용을 답한다
완전성 (Completeness)	근거에 있는 필요한 항목을 모두 담았다	일부 조건이나 예외가 빠졌다	핵심 항목이 빠졌다

두 기준을 한 점수로 합치면 어느 쪽이 부족한지 알 수 없다. 기준별 점수를 따로 남긴다.



일관성·형식 준수 루브릭 설계

앞뒤 모순

같은 답 안에서 어긋나는 진술이 있는지 본다.
숫자·날짜·주체가 문장마다 달라지는 경우가 대표적이다.

설명의 비약

근거에서 결론으로 넘어가는 중간 단계가 빠졌는지 본다. 결론만 맞아도 설명이 이어지지 않으면 감점한다.

출처 표기

근거로 쓴 청크를 표기했는지 확인한다. 표기 형식은 LLM 판정 없이 규칙으로도 검사할 수 있다.

길이·형식

요구한 형식과 길이를 지켰는지 본다. 파서로 먼저 걸러 내면 LLM 호출을 줄일 수 있다.

금지 표현

추측이나 단정 표현을 쓰지 않았는지 본다. 표현 목록을 정해 두고 검사한다.

형식 준수는 규칙으로 검사할 수 있는 부분이 많다.
규칙으로 걸러지는 항목은 규칙 단계에서 처리해 LLM 채점 비용을 줄인다.



네 기준의 종합과 가중치

기준별 점수를 그대로 쓰지, 하나로 묶을지는 그 점수로 무엇을 결정하는지에 따라 달라진다.

기준별 점수

관련성·일관성·충실성·완전성을
각각 1~5점으로 받는다

루브릭이 기준마다 다르므로
점수도 기준별로 남긴다



종합 방식

가중 평균 · 서비스에서 중요한
기준에 가중치를 준다

하한 · 양보할 수 없는 기준에는
최소 점수를 따로 둔다



판정

게이트에는 종합 점수와
하한을 함께 넣는다

차단될 때는 어느 기준에서
막혔는지 함께 기록한다

실습에서 충실성 하나만 구현하는 이유는 충실성이 RAG에서 가장 먼저 확인할 기준이기 때문이다.
나머지 기준도 루브릭만 바꾸면 같은 구조로 채점한다.



Faithfulness Judge 구현 코드

Judge 구현

```
class JudgeResult(BaseModel):
    reason: str = Field(description="판정 근거")
    score: int = Field(ge=1, le=5)

JUDGE_SYS = (
    "너는 RAG 답변의 Faithfulness 평가자다.\n"
    "5점: 모든 주장이 컨텍스트에서 확인됨\n"
    "3점: 일부가 컨텍스트 밖의 내용\n"
    "1점: 핵심 주장이 모순되거나 지어낸 내용\n"
    "먼저 근거를 서술한 뒤 점수를 매긴다.")

def judge(context, question, answer):
    resp = client.chat.completions.parse(
        model="gpt-5.4", temperature=0,
        messages=[...],
        response_format=JudgeResult)
    return resp.choices[0].message.parsed
```

- 척도(5·3·1점)를 문장으로 정의한다. 정의가 없으면 점수 해석이 매번 달라진다.
- 구조화 출력으로 reason과 score를 강제한다. 범위 이탈과 파싱 실패를 방지한다.
- temperature=0에도 점수는 실행마다 달라진다. 실습에서 확인한다.

Judge의 품질은 모델보다 루브릭과 출력 구조 설계의 영향을 크게 받는다.



채점 모델은 성능만으로 고르지 않는다. 자기 답에 후한 경향과 비용을 함께 고려한다.

1 판정 모델 분리 generator ≠ judge

답 생성 모델과 채점 모델을
다르게 둔다

확인

같은 모델이면 교차 채점을 함께 본다

2 자기선호 self-preference

자기 답에 후한 점수를 준다

완화

작성자 정보를 제거하고 채점한다

3 모델 급 capability

작은 모델은 편향과 변동이
크다

사람 표본 일치율로 검증

모델 버전 고정

4 비용·속도 cost

호출은 문항 수 × 기준 수로
늘어난다

설계

상시 채점은 표본, 전체는 배치

채점 모델을 바꾸면 점수의 절대 수준이 달라진다. 모델과 버전을 기록하고, 바꿀 때는 기준선을 다시 측정한다.



Judge의 4대 편향

Judge도 LLM이다. 네 가지 편향을 알아야 결과를 해석할 수 있다.

01

위치 편향

쌍대 비교에서 먼저 제시된 답을 선호
처방: 순서 교체 이중 판정

02

장황함 편향

길고 상세해 보이는 답에 후한 점수
처방: 길이 아닌 근거 중심 루브릭

03

자기선호 편향

자기(같은 모델)가 쓴 답을 선호
처방: 다른 모델·교차 채점

04

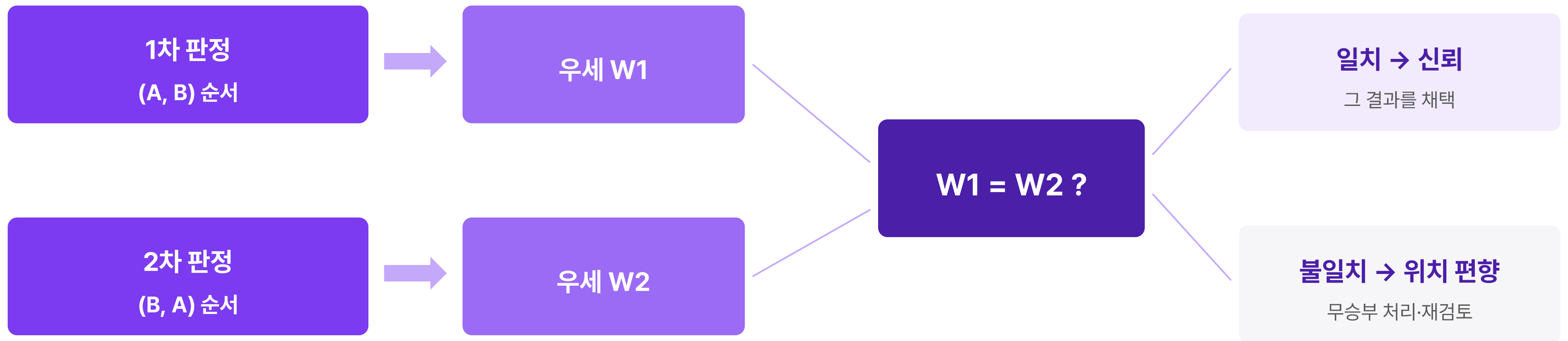
점수 불안정

같은 입력에도 실행마다 점수 변동
처방: 반복 채점·표준편차 확인

점수 불안정과 위치 편향은 실습에서 직접 재현한다. 순서만 바뀌도 우열이 바뀌는 것을 확인한다.

순서 교체 이중 판정 (swap & re-judge)

쌍대 비교는 순서를 바꿔 두 번 판정하고, 결과가 일치할 때만 신뢰한다.



잘못된 결론을 막는 대가로 보면 두 번 판정하는 비용은 저렴하다.



편향 완화 기법 5종

기준 명세화

척도를 점수별 문장으로 정의한다. 5·3·1점의 조건을 명시하면 해석 변동이 줄어든다.

few-shot 예시

채점 예시를 넣어 기준점을 고정한다. 실습에서 표준편차가 줄어드는 것을 확인한다.

쌍대 비교

절대 점수 대신 두 답변의 상대 비교를 사용한다. 점수 척도 자체의 불안정을 우회한다.

순서 교체

쌍대 비교는 (A,B)·(B,A) 두 번 판정해 일치할 때만 채택한다.

사람 채점 보정

Judge 점수를 사람 채점 표본과 대조해 일치율을 확인한다. 최종 신뢰의 근거이다.

완화 기법은 모두 실습에서 코드로 확인한다. 최종 보정은 사람이 직접 수행한다.



Judge 신뢰성 검증 절차

Judge를 채점에 투입하기 전에 사람 채점 표본과 얼마나 맞는지 확인한다.

20~50건

사람이 직접 채점할 표본

핵심 동작을 포괄하는 표본에서 시작한다. 통과·실패 라벨이 한쪽으로 쏠리지 않게 고른다

$K \cdot \rho$

어떤 통계로 볼 것인가

통과·실패 같은 이진 판정은 카파, 1~5점 척도는 가중 카파나 순위 상관을 쓴다

0.61+

합의 수준의 눈금

카파 0.61~0.80을 상당한 일치로 보는 관례가 있다. 미달이면 루브릭을 수정해 다시 측정한다

사람 표본 채점은 Judge 도입 이후에도 남겨 둔다. 루브릭이나 채점 모델을 바꿀 때마다 일치율을 다시 확인한다.



절대 채점과 상대 비교의 선택

두 방식은 대체 관계가 아니다. 그 점수로 무엇을 결정하려는지에 따라 고른다.

구분	절대 채점	상대 비교
산출물	문항별 1~5점	두 답변 중 우세한 쪽
강점	절대 점수가 남으므로 게이트 기준으로 바로 쓸 수 있다	미세한 차이를 비교적 잘 구분한다
약점	점수가 한 값에 몰리고 버전 사이 비교가 흔들린다	순서 편향이 있고 비교 횟수가 조합 수만큼 늘어난다
쓰는 곳	상시 모니터링과 게이트 판정	두 구성 중 하나를 고르는 A/B

어느 방식이든 판정 근거를 함께 받아 두면 결과를 나중에 다시 확인할 수 있다.



거절과 범위 밖 질문의 채점

문서에 답이 없는 질문에서는 "모른다"가 정답이다. 답변 유무와 근거 유무를 함께 보아야 판정이 갈린다.

	근거가 있는 질문	근거가 없는 질문
답을 한다 (답변 생성)	정상 근거에 충실하면 통과 충실성·완전성으로 채점	환각 없는 내용을 만들어 답한다 가장 무겁게 감점
모른다고 한다 (거절)	과도한 거절 찾을 수 있는 답을 놓쳤다 검색 실패와 생성 실패를 분리	정상 거절 이 칸이 정답이다 거절도 점수로 인정

거절은 별도 지표로 집계한다. 정답률만 보면 거절을 늘려 환각을 줄인 구성이 좋아 보인다.

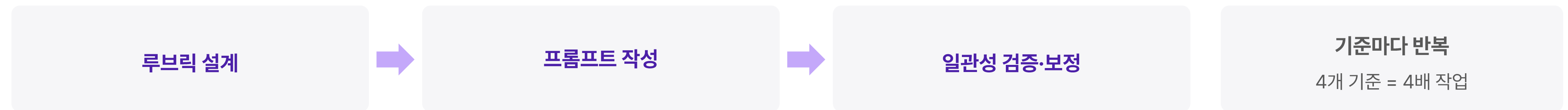
RAGAS를 활용한 종합 평가



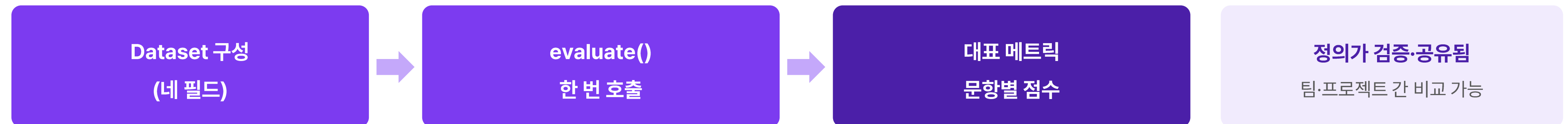
RAGAS 프레임워크

기준마다 Judge를 새로 만드는 대신, 검증된 메트릭 파이프라인 하나로 표준화한다.

직접 만든 Judge



RAGAS 방식 · ragas 0.2.x



내부에서는 여전히 Judge LLM과 임베딩이 동작하지만, 프롬프트·집계는 프레임워크가 담당한다. 실습에서는 gpt-5.4를 evaluate 인자로 주입한다.

직접 만든 Judge가 필요 없어지는 것은 아니다. 공개된 정의에 없는 기준(자사 정책 준수 등)은 실습 7의 방법으로 만든다.

RAGAS 메트릭의 구성과 다루는 범위

생성 품질을 측정하는 메트릭은 하나로 정해져 있지 않다. RAGAS는 공개된 정의를 용도별 카테고리로 묶어 제공한다.

RAG · 검색과 생성

Context Precision·Recall, Faithfulness 등. 여기서 다루는 카테고리이다.

범용 루브릭

Aspect Critic, Rubrics-based Scoring 등. 채점 기준을 직접 정의한다.

자연어 비교

Factual Correctness, BLEU·ROUGE 등. 정답지와 문 자연·의미를 대조한다.

Nvidia 메트릭

Answer Accuracy, Response Groundedness 등. 같은 용도의 다른 정의이다.

에이전트·도구

Tool Call Accuracy, Agent Goal Accuracy 등. 도구 호출과 목표 달성을 본다.

태스크 특화

SQL 질의 동등성, 요약 품질 등. 출력 형식이 정해진 태스크에 쓴다.

분류 기준은 두 가지이다. 한 턴을 채점하는지 여러 턴을 채점하는지, 그리고 LLM 기반인지 아닌지이다.
여기서 다루는 것은 RAG 카테고리의 대표 메트릭이다.



RAG 대표 메트릭 정리

검색 품질 둘과 답변 품질 셋을 정의로 정리하고, 정답지(reference)를 요구하는 메트릭을 구분한다.

Context Precision

관련 청크가 상위에 있는가. 청크별 precision@k의 가중 평균이다.

Context Recall

정답(reference)의 주장 중 검색 근거로 뒷받침되는 비율. 정답지가 필요하다.

Faithfulness

답변의 주장이 검색된 근거에서 확인되는가. 환각 탐지의 핵심 기준이다.

Answer Relevancy

답이 질문에 얼마나 부합하는가.
질문·답변 임베딩 유사도 기반으로 계산한다.

Answer Correctness

답변이 정답지와 사실로 일치하는가. 사실 일치 F1과 의미 유사도의 가중 평균이다.

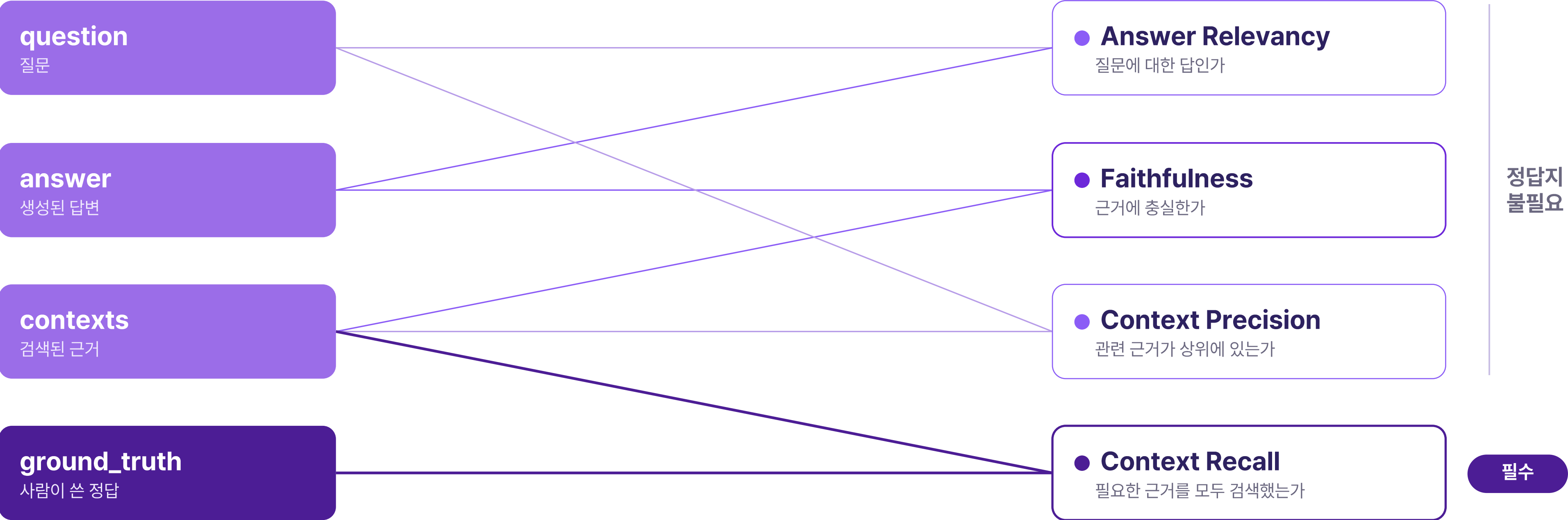
정답지 요구 여부

Context Recall과 Answer Correctness는 정답지가 필요하고, 나머지 셋은 정답지 없이 계산한다.



데이터 필드와 메트릭의 대응

4개 필드 중 무엇을 사용하는지가 메트릭의 특징을 가른다. 정답지 요구 여부에 따라 메트릭을 나눈다.



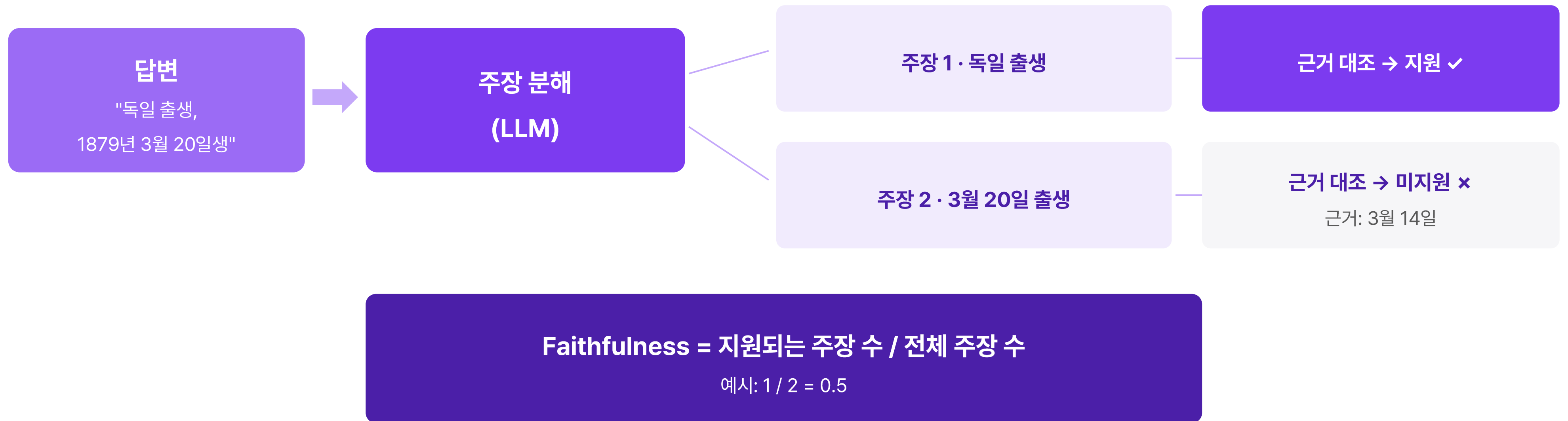
이 대응 관계에서는 Context Recall이 정답지를 요구한다. Answer Correctness도 정답지를 쓰므로, Golden Dataset이 필요한 이유가 된다.

선 굵기: ground_truth 연결선이 이 도식의 핵심이다.



Faithfulness의 계산 원리

답변을 주장 단위로 나누고, 근거에서 확인되는 주장의 비율이 점수가 된다.

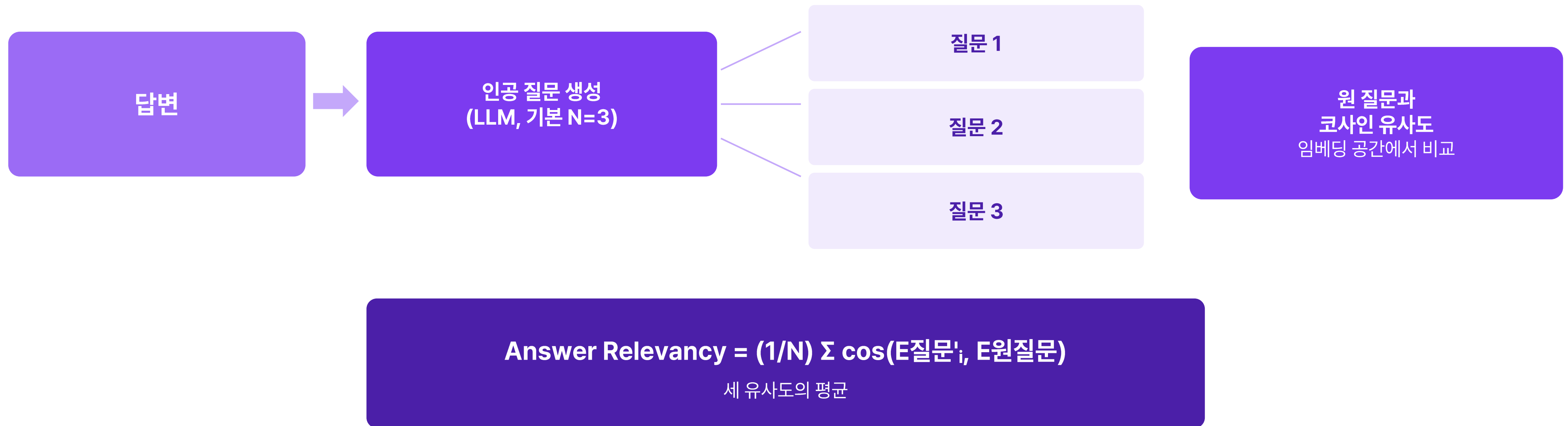


Judge는 문장 전체에 점수를 매기지만, RAGAS는 주장 단위로 나누어 근거 충실성을 계산한다.



Answer Relevancy의 계산 원리

답에서 질문을 역으로 생성해, 원래 질문과의 유사도를 임베딩으로 측정한다.



동문서답이면 역질문이 원 질문과 멀어져 점수가 떨어진다. 불완전하거나 군더더기가 많은 답도 감점된다.

다만 사실 정확성은 보지 않으며, 이는 Faithfulness가 담당한다.

코사인 특성상 0~1이 보장되지는 않는다.

순위마다 precision@k를 구하고, 관련 청크가 나온 순위의 값만 평균한다.

순위	검색된 청크 (Top-3)	관련 판정 v_k	Precision@k	점수 기여
1위	d03-p09-c01 재구매 LTV 4.8배	관련 ✓ (1)	$1 / 1 = 1.00$	$1.00 \times 1 = 1.00$
2위	d02-p45-c00 AI 상품추천	무관 × (0)	$1 / 2 = 0.50$	$0.50 \times 0 = 0$
3위	d03-p09-c02 LTV 용어 정의	관련 ✓ (1)	$2 / 3 = 0.67$	$0.67 \times 1 = 0.67$
합계	관련 청크 2개	$\sum v_k = 2$	분자 = 1.67	$1.67 \div 2 = 0.83$

Context Precision = $\sum (\text{Precision@k} \times v_k) \div \text{관련 청크 수}$

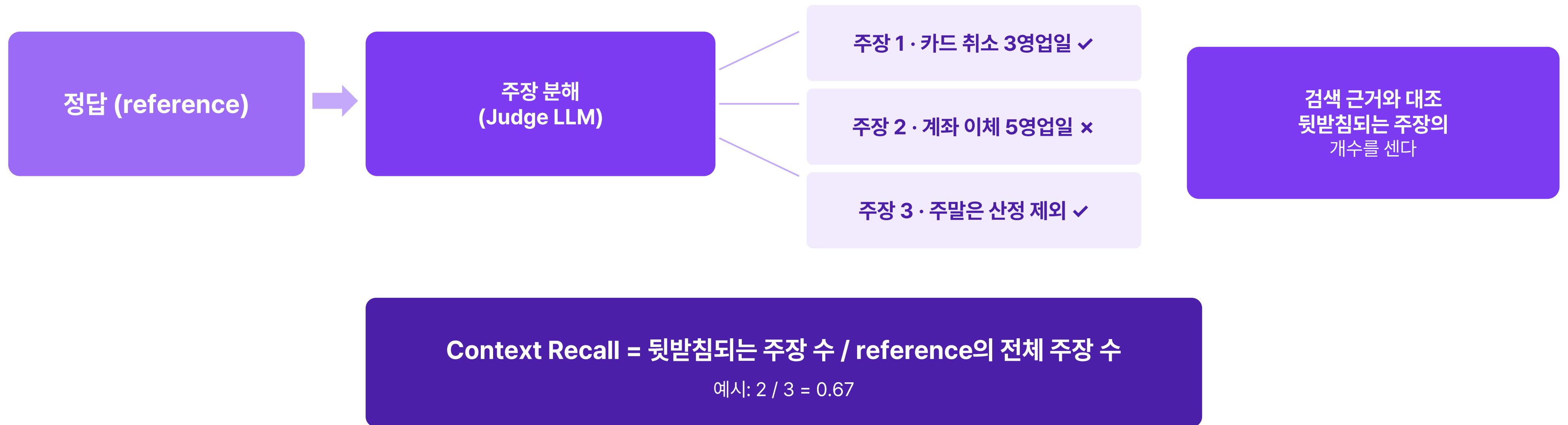
예시: $1.67 / 2 = 0.83$

정답 청크 2개를 맞혀도 순서가 ✓✓×면 1.00, ✓×✓면 0.83, ×✓✓면 0.58이 된다.
Precision@K(관련 문서 수 ÷ K)와 달리 순위가 점수에 반영된다.



Context Recall의 계산 원리

정답(reference)을 주장 단위로 나누고, 검색 근거에서 뒷받침되는 주장의 비율이 점수가 된다.



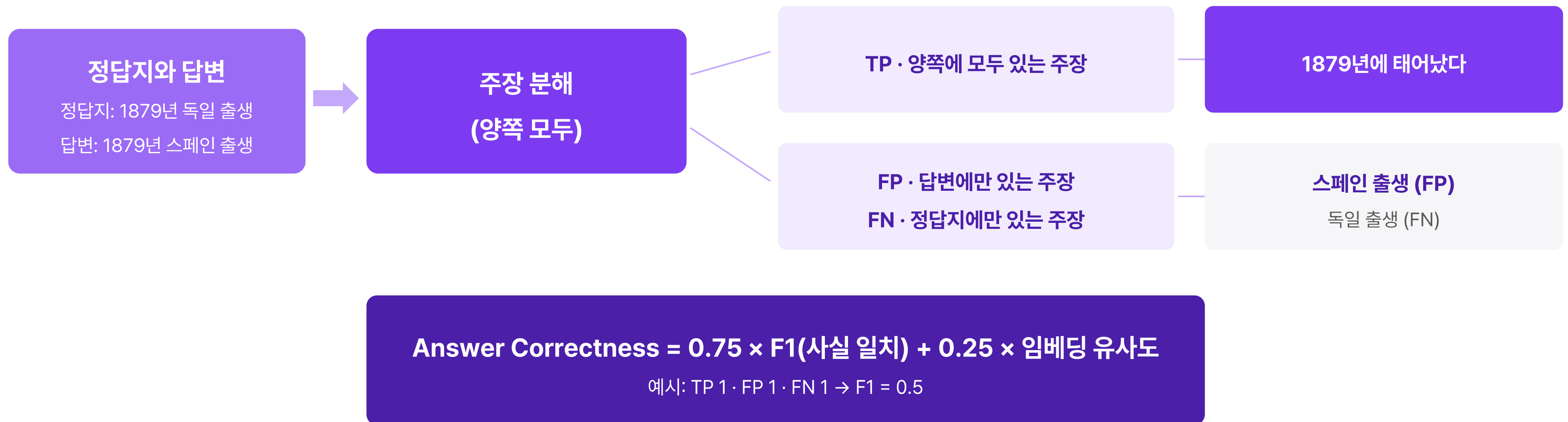
분모가 reference의 주장 수이므로, reference가 없으면 분모를 만들 수 없다.

Context Recall이 정답지를 요구하는 메트릭인 이유이다.

ragas 0.2.x의 필드 이름은 reference이며, 0.1.x의 ground_truths는 사용하지 않는다.

Answer Correctness의 계산 원리

정답지가 있을 때 쓰는 대표 메트릭이다. 답변과 정답지를 주장 단위로 나눠 겹침과 어긋남을 센다.



충실성은 검색된 근거와 대조하고, 이 메트릭은 정답지와 대조한다. 근거에 충실해도 정답지와 다르면 점수가 내려간다.



평가 데이터셋 준비

필드 매핑

```
from ragas import EvaluationDataset
from ragas.dataset_schema import SingleTurnSample

samples = []
for item in eval_set:
    ctx = retriever.invoke(item["question"])
    answer = rag_chain.invoke(item["question"])
    samples.append(SingleTurnSample(
        user_input=item["question"],
        retrieved_contexts=[d.page_content for d in ctx],
        response=answer,
        reference=item["answer"],
    ))

dataset = EvaluationDataset(samples=samples)
```

- 네 필드는 파이프라인 출력에서 그대로 채운다. 질문, 검색된 청크 목록, 생성된 답변, 사람이 쓴 정답이다.
- 검색 근거는 문자열 목록으로 넣는다. 하나로 이어 붙이면 청크 단위 판정이 성립하지 않아 Context Precision 값이 달라진다.
- 정답지는 Context Recall과 Answer Correctness에만 필요하다. 정답지가 없으면 이 두 메트릭은 계산할 수 없다.
- 메트릭마다 요구하는 필드가 다르므로, 쓸 메트릭을 먼저 정한 뒤 데이터를 구성한다.

필드가 빠지면 그 메트릭은 계산되지 않는다. 데이터셋을 만든 뒤 한 문항으로 먼저 실행해 확인한다.



evaluate() 호출 코드

RAGAS 평가

```
from ragas import evaluate
from ragas.metrics import (faithfulness,
                             answer_relevancy, context_precision,
                             context_recall)

judge_llm = LangchainLLMWrapper(ChatOpenAI(
    model="gpt-5.4", base_url=BASE_URL,
    api_key=OPENAI_API_KEY, temperature=0))
judge_emb = LangchainEmbeddingsWrapper(
    OpenAIEmbeddings(model="text-embedding-3-small",
                      base_url=BASE_URL, api_key=OPENAI_API_KEY))

result = evaluate(ds,
                  metrics=[faithfulness, answer_relevancy,
                           context_precision, context_recall],
                  llm=judge_llm, embeddings=judge_emb)
df = result.to_pandas()
```

- Judge LLM·임베딩을 인자로 넘겨 Elice 엔드포인트를 쓰게 한다.
- 호출 수는 문항 수 × 메트릭 수이다. 작은 평가셋으로 먼저 확인한다.
- 결과의 질문 컬럼명은 user_input, 답변은 response이다.

evaluate 한 번이면 지정한 메트릭이 문항별로 나온다. 남은 일은 그 숫자를 해석하는 것이다.

호출 수는 문항 수와 메트릭 수, 그리고 검색 청크 수에 따라 늘어난다.

메트릭	문항당 LLM 호출	임베딩 호출	호출 수를 늘리는 요인
Faithfulness	2회	없음	주장 추출 1회 + 근거 대조 1회로 묶여 있다
Answer Relevancy	3회	2회	생성하는 역질문 수(기본 3)에 비례한다
Context Precision	K회	없음	청크마다 한 번씩 판정해 K에 비례한다
Context Recall	1회	없음	정답지 주장 수가 많아도 호출은 한 번이다

K=3에서 이 네 메트릭을 함께 쓰면 문항당 LLM 호출이 9회 수준이다.
100문항이면 900회에 이르므로 작은 평가셋으로 먼저 확인한다.



점수 패턴 진단 매트릭스

점수는 순위를 매기는 자료가 아니라 원인을 찾는 자료이다. 두 지표의 조합이 실패 유형을 가리킨다.

	Context Recall 낮음	Context Recall 높음
Faithfulness 높음	검색 실패 근거를 검색하지 못했는데 답변은 그럴듯함 처방: 인덱싱·검색 개선	정상 Relevancy까지 높으면 통과 낮으면 동문서답 의심
Faithfulness 낮음	검색·생성 모두 문제 검색부터 개선하고 재평가 처방: 검색 개선 후 생성 개선	생성 실패 근거는 검색했는데 답변이 벗어남 처방: 프롬프트·모델 개선

실습에서는 기준선을 0.7로 둔다. 기준선은 서비스 요구사항에 따라 조정한다.
경계값 근처의 문항은 점수만 보지 말고 원문을 열어 확인한다.



메트릭 조합 해석 사례

여러 메트릭을 함께 읽으면 한 지표만으로는 좁혀지지 않던 원인 후보가 줄어든다.

1 Precision 정밀도만 낮음 무관한 청크가 섞여 든다 확인 절차 상위 K개의 순서 확인	2 Recall 회수만 낮음 필요한 근거가 빠져 있다 확인 절차 색인에 정답 문서가 있는지	3 Faithfulness 충실성만 낮음 근거 밖 문장이 섞인다 확인 절차 주장 단위로 근거 대조	4 Correctness 정확성만 낮음 정답지와 내용이 다르다 확인 절차 근거 문서 자체를 검증
---	---	---	--

조합이 가리키는 것은 원인 후보이다. 해당 문항의 질문과 검색 청크, 답변 원문을 함께 열어 확인하는 것을 권장한다.



진단별 처방과 개선 백로그

진단	점수 신호	처방	백로그 기재 사항
검색 실패	Recall 낮음 · Faithfulness 높음	청킹·임베딩·쿼리 개선	실패 질문·농친 근거
생성 실패	Recall 높음 · Faithfulness 낮음	프롬프트 강화·모델 교체	환각 주장 원문
모두 문제	Recall·Faithfulness 모두 낮음	검색 개선 후 재평가	우선순위 최상
동문서답	Relevancy만 낮음	답변 형식 지시 추가	질문 유형

진단을 빈도순으로 집계하면 개선 백로그가 된다. 가장 잦은 유형부터 개선하면 효과가 먼저 드러난다.

나누는 기준은 도구가 아니라 정의의 출처이다. 공개 정의와 자사 정의는 포괄 범위가 다르다.

공개 정의 · 정의와 계산이 공개된 메트릭

근거 충실성
답변이 근거 안에 있는가

질문 적합성
질문에 답하고 있는가

검색 품질
필요한 근거가 들어왔는가

정의가 공개되어 있다
팀·프로젝트 간 비교 가능

자사 정의 · 직접 설계한 루브릭

정책 준수
금지 표현·면책 문구

형식 준수
항목 수·인용 표기 규칙

통과 어조
고객 응대 문체 규정

자체 서비스에만 있는 기준
RAGAS의 범용 루브릭으로도 구현할 수 있다

두 결과는 같은 평가셋 위에서 함께 집계한다. 공개 정의 점수와 자사 기준 통과율을 한 표에 두면 릴리즈 판단에 함께 쓸 수 있다.

공개 정의로 기준선을 먼저 세우고, 비어 있는 부분을 자사 기준으로 보완하는 순서가 적절하다.



//

점수는 단건이 아니라 추이로 판단한다

//

LLM 채점은 일정 수준의 편차가 있다. 0.72와 0.74의 비교는 통계 검정으로 판단하는 것을 권장한다.

단건 점수 차이는 결론이 아니라 신호이다. 같은 평가셋으로 기록한 추이와 통계 검정이 판단의 근거가 된다.



점수 변동성 관리

채점 결과는 실행마다 달라진다. 변동 폭을 알아야 점수 차이를 해석할 수 있다.

0

temperature 0에서도 남는 변동

같은 입력의 채점이 완전히 같게 나오지는 않는다. 모델 갱신도 변동 요인이다

3회

같은 조건에서 반복 측정

세 번 이상 실행해 중위수와 변동 폭을 함께 기록하면 단발 편차에 덜 흔들린다

4개

재현을 위해 남기는 기록

모델 버전 · 프롬프트 · 평가셋 버전 · 샘플링 설정

변동 폭보다 작은 점수 차이는 개선으로 보기 어렵다. 폭을 먼저 측정한 뒤 판단하는 것을 권장한다.



인용·출처 표기 평가

점수는 팀이 보는 숫자이고, 사용자가 보는 것은 답변과 그 출처 표기이다.

1단계 · 표기 여부

답변에 출처 표기가 있는가
규칙으로 판별 가능

검출 대상: 표기 누락



2단계 · 표기 정확도

가리킨 청크에 그 내용이 있는가
청크 대조로 판별

검출 대상: 잘못된 청크 지목



3단계 · 표기 완전성

근거로 쓴 청크를 모두 밝혔는가
주장별 근거 대조

검출 대상: 일부 근거 누락

충실성은 답변이 근거 안에 있는지를 보고, 인용 평가는 그 근거를 올바르게 가리켰는지를 본다.

평가셋 구축: Golden Dataset



Golden Dataset의 세 요소

KEY 01

질문

- 실사용자의 말투로 쓴 질문이다. 문서 문장을 그대로 옮기면 검색 지표가 실제보다 높게 나온다.

KEY 02

정답

- 근거 청크의 내용만으로 쓴 정답이다. Context Recall의 reference가 된다.

KEY 03

근거

- 정답의 출처인 청크 ID이다. Retrieval Metrics(실습 2)의 정답 집합이 된다.

이 셋이 갖춰지면 측정의 기준선으로 쓸 수 있다. 기준선이 흔들리면 비교 결과도 함께 흔들린다.



평가셋 규모와 결과의 신뢰도

문항이 적으면 같은 점수라도 폭이 넓다. 폭을 알고 나서야 차이를 해석할 수 있다.

1 문항이 적으면 폭이 넓다

10문항에서 8개 통과면 80%이지만, 95% 신뢰구간은 약 49%~94%이다.

이 폭 안에서는 60%인 시스템과 90%인 시스템을 구분할 수 없다.

실습 평가셋



14~38문항

2 문항을 늘리면 폭이 좁아진다

같은 80%에서 30문항이면 약 63%~91%, 100문항이면 약 71%~87%이다.

폭은 문항 수의 제곱근에 반비례해 줄어든다. 문항을 네 배로 늘리면 폭이 절반이 된다.

10

30

100

300

문항 수

3 단건 차이로 결론 내지 않는다

두 설정의 차이가 폭 안에 들어가면 개선으로 보기 어렵다.

같은 문항으로 짝지어 비교하면 같은 문항 수에서도 더 작은 차이를 판별할 수 있다.

폭 $\propto 1/\sqrt{n}$

판별력 상승

실습 평가셋은 구조를 익히기 위한 규모이다. 운영 판정에는 문항을 늘리고 통계 검정을 함께 적용한다.

필요한 필드는 무엇을 채점할지에 따라 달라진다. 검색 평가와 생성 평가의 요구가 다르다.

필드	예시 값	무엇에 쓰이는가	평가 대상	없으면 불가능한 일
question	재구매 고객의 LTV는 신규 대비 몇 배인가	파이프라인 입력	공통	측정 자체가 불가
answer	신규구매 고객 대비 4.8배 높다	생성 채점의 정답지	생성	Context Recall · Answer Correctness
evidence_ids	["d03-p09"]	검색 적중 판정의 정답 집합	검색	Recall@K · MRR · NDCG
meta	domain · question_type · context_type · source · version	실패를 유형별로 분리	공통	원인별 집계

필드 이름을 먼저 고정한 뒤 데이터를 채운다. 이름이 흔들리면 도구를 옮길 때마다 변환 코드를 새로 써야 한다.



공개 데이터셋의 파일 구조

표준 세 파일

```
# corpus.jsonl - 문서 한 건이 한 줄
{"_id": "d03-p09", "title": "Dighty 이커머스 데이터 인사이트",
 "text": "재구매 고객의 평균 LTV, 신규고객보다 4.8배 ..."}

# queries.jsonl - 질문 한 건이 한 줄
{"_id": "q11",
 "text": "재구매 고객의 LTV는 신규 대비 몇 배인가"}

# qrels/test.tsv - 탭 구분, 첫 행은 헤더
query-id    corpus-id    score
q11         d03-p09      1
```

- 코퍼스와 질문을 나눠 두면 같은 코퍼스에 여러 질문 세트를 붙일 수 있다.
- qrels는 질문 ID, 문서 ID, 관련도 점수 세 열이다. 여기에 없는 조합은 관련 없음으로 취급한다.
- TREC 원형은 헤더 없이 네 열을 쓴다. 이 형식은 그중 iteration 열을 빼고 헤더를 붙인 형태이다.
- 자체 평가셋도 같은 구조로 저장하면 공개 데이터셋과 같은 코드로 지표를 계산할 수 있다.

근거를 문서 ID로 가리키는 방식이 공개 검색 데이터셋의 공통 규칙이다.
근거 본문을 파일 안에 그대로 포함하는 평가 프레임워크 형식과는 저장 방식이 다르다.



관련도 라벨: 이진과 등급

qrels의 score 열에 무엇을 넣는지에 따라 계산할 수 있는 지표가 달라진다.

이진 라벨 · 0 또는 1

- 관련 있음과 없음만 구분한다
- MS MARCO · Natural Questions · HotpotQA · FiQA · SciFact가 이 방식이다
- 질문당 정답 문서가 한두 건인 데이터셋에 주로 쓰인다

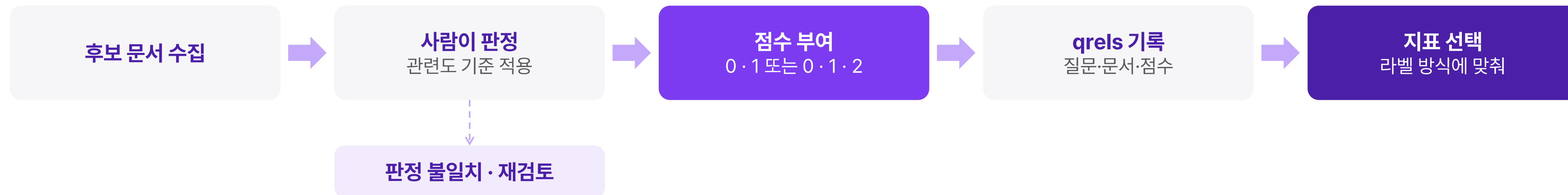
계산 가능 · Recall@K · Precision@K · MRR

등급 라벨 · 0 · 1 · 2

- 부분적으로 관련 있는 문서에 중간 점수를 준다
- TREC-COVID · NFCorpus · DBPedia가 이 방식이다
- 상위 순위에 더 관련 있는 문서가 왔는지까지 구분할 수 있다

계산 가능 · NDCG@K

라벨을 만드는 순서



한 질문에 근거가 여러 개인 데이터셋도 있다. 문장 단위로 근거에 라벨을 붙이는 방식은 문서 단위보다 한 단계 세밀하다.

널리 쓰이는 검색 평가 데이터셋은 규모와 라벨 방식이 서로 다르다.

데이터셋	코퍼스	쿼리	라벨 · 질문당 근거 수	주의
MS MARCO passage	880만	6,980 (dev)	이진 · 약 1.1	비상업 연구용
Natural Questions	268만	3,452	이진 · 약 1.2	위키 기반 일반 지식
HotpotQA	523만	7,405	이진 · 약 2.0	CC BY-SA 4.0 · 멀티 홉
FiQA-2018	57,638	648 (test)	이진 · 약 2.6	CC BY-SA 4.0 · 금융 도메인
SciFact	5,183	300 (test)	이진 · 약 1.1	CC BY-SA 4.0 · 과학 주장 검증

모두 같은 세 파일 구조를 쓰므로 지표 계산 코드를 그대로 재사용할 수 있다. 규모가 큰 데이터셋은 일부만 내려받아 쓰는 방법도 있다.

한국어 평가에 바로 쓸 수 있는 공개 데이터셋도 있다. 라이선스 조건을 함께 확인한다.

데이터셋	규모	구조	라이선스	용도
Allganize RAG 평가 (한국어)	5개 도메인 × 60문항 = 300문항	질문 · 정답 답변 · 근거 문서명 · 쪽 번호 · 근거 형태	MIT	검색 · 생성
MIRACL 한국어	쿼리 868 (train) · 213 (dev)	질문마다 관련 문단 목록과 비관련 문단 목록	Apache-2.0	검색
Ko-StrategyQA	코퍼스 27.8k · 쿼리 8.5k	표준 세 파일 형식 · 멀티 홉	공개	멀티 홉 검색
KorQuAD 1.0	학습 60.4k · 검증 5.77k	지문 · 질문 · 정답 위치	CC BY-ND 4.0	독해 · 생성

KorQuAD 1.0은 변경 금지 조건이 붙어 있어 문항을 수정해 쓰기 어렵다.
도메인이 한정된 데이터셋은 모델 비교용으로 쓰는 것을 권장한다.



Allganize 데이터셋의 레코드 구조

레코드 한 행을 필드 단위로 살펴보면 어떤 평가가 가능한지 드러난다.

레코드 한 행 (commerce 도메인 · q11)

```
domain      : "commerce"
question    : "재구매 고객의 평균 LTV는 신규 대비 몇 배인가"
target_answer : "신규구매 고객 대비 4.8배 높다"
target_file_name : "Dighty Data Insight Report 2023.pdf"
target_page_no : 9
context_type : "paragraph"
```

시스템별 답변 · 정답 여부 열

target_file_name

정답이 있는 문서를 지정한다. 검색 결과에 이 문서가 들어왔는지로 적중을 판정할 수 있다.

target_page_no

쪽 번호까지 라벨이 붙어 있어 문서 단위보다 좁은 범위로 적중을 볼 수 있다.

target_answer

정답 답변이 있어 생성 채점의 정답지로 쓸 수 있다.

context_type

근거가 문단·표·이미지 중 무엇인지 표시된다. 근거 형태별로 실패를 분리할 수 있다.

근거 형태 분포는 전체 300문항 기준 문단 148 · 이미지 57 · 표 50 · 텍스트 45이고,

실습 코퍼스인 commerce 60문항은 문단 38 · 이미지 17 · 표 5이다.

공개 리더보드는 생성 정답률만 제공하므로 검색 적중률은 직접 계산해야 한다.

공개 데이터셋으로 확인할 수 있는 것과 확인할 수 없는 것이 나뉜다.

1 도메인 차이

질문과 문서가 자체 서비스의
것이 아니다. 용어와 표현이
다르게 쓰인다.

확인 가능

모델 비교

확인 어려움

자체 문서에서의 성능

2 문서 형식 차이

표·이미지·서식이 섞인 사내
문서와 정제된 코퍼스는
구조가 다르다.

확인 가능

기법 비교

확인 어려움

파싱·청킹 단계의 실패

3 라벨 부재

사내 코퍼스에는 정답 근거
라벨이 처음부터 없다.
직접 만들어야 한다.

확인 가능

공개 코퍼스

확인 어려움

자체 코퍼스의 정답 집합
라벨 없이는 측정 불가

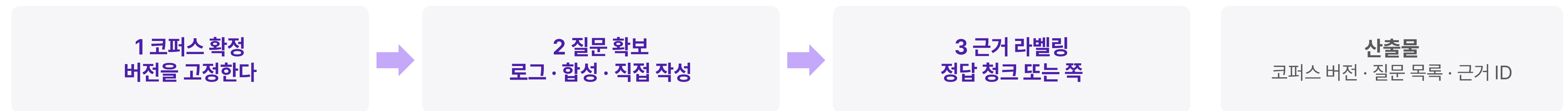
공개 데이터셋은 모델과 기법을 비교하는 자료로 쓰고, 서비스 배포 판정은 자체 평가셋으로 하는 구성을 권장한다.



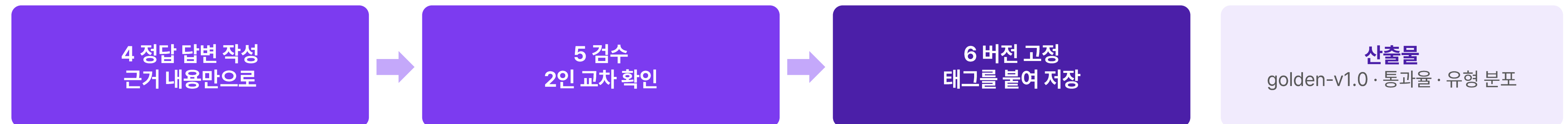
자체 평가셋 구축 절차

6단계로 나누면 단계마다 무엇이 산출되는지 분명해진다.

1~3단계 · 문항의 재료를 만든다



4~6단계 · 문항을 완성하고 고정한다



관측 도구 문서는 핵심 동작을 포괄하는 20~50문항으로 시작해, 회귀 세트를 100~1,000문항까지 늘리는 기준을 제시한다.

평가 프레임워크 문서와 검색 벤치마크 논문에는 권고 최소 문항 수가 없다. 서비스의 질문 유형 수를 기준으로 정하는 것이 현실적이다.



질문 확보와 유형 분포 설계

질문은 실사용 로그, 합성 생성, 직접 작성으로 모은다. 유형 분포를 먼저 정한다.

1 Factual 사실 확인 정의와 수치를 묻는다 채점 시 유의점 근거 한 곳에서 확인	2 Procedural 절차 순서와 방법을 묻는다 채점 시 유의점 여러 단계의 근거 필요	3 Comparative 비교 두 대상의 차이를 묻는다 채점 시 유의점 근거 둘 이상 필요	4 Out-of-scope 범위 밖 문서에 답이 없는 질문 채점 시 유의점 모른다가 정답
---	--	---	--

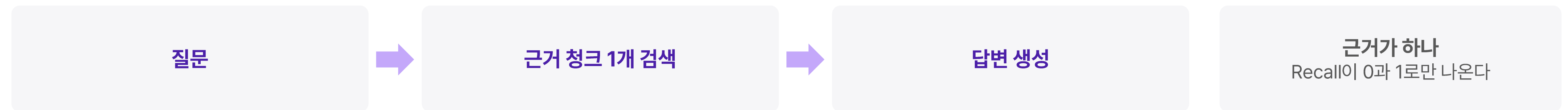
실사용 로그의 실패 질문을 먼저 문항으로 만들고, 부족한 유형을 합성과 직접 작성으로 채우는 순서를 권장한다.



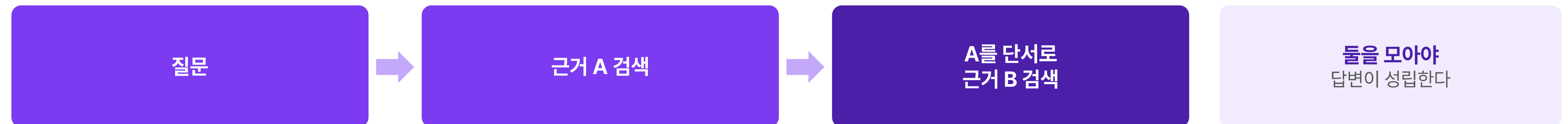
단일 흡과 멀티 흡 질문

여러 문서를 이어야 답이 나오는 질문은 근거가 둘 이상이므로 채점 방식부터 달라진다.

단일 흡 · 지금까지 다룬 평가셋



멀티 흡 · 근거를 이어야 답이 나온다



예) 단일 흡은 "재구매 LTV는 신규의 몇 배인가", 멀티 흡은 "로열 고객 비중이 높은 업종의 재구매 전략은"처럼 두 쪽을 이어야 답이 되는 질문이다.

근거가 둘 이상이면 Recall이 부분 점수로 나오고, 중간 한 단계만 틀려도 최종 답이 어긋난다. 이 강의에서는 개념 수준까지 다룬다.



합성 평가셋 생성 파이프라인

평가셋은 정해진 절차를 거쳐 만든다. LLM이 생성하고, 2단계 필터로 거르고, 버전을 붙여 확정한다.



문항의 세 요소

질문

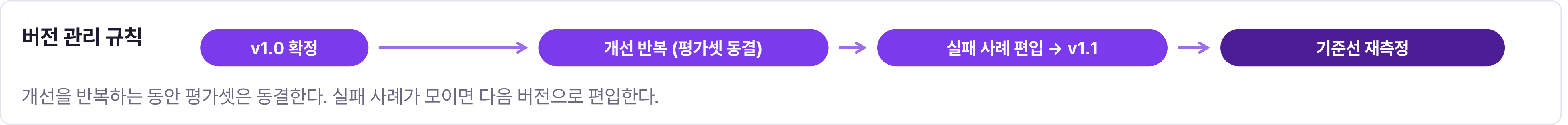
실사용자의 말투 · 문서 문장을 그대로 옮기면 검색 지표가 실제보다 높게 나옴

정답

청크 내용만으로 작성 · Context Recall의 reference

근거 청크 ID

Retrieval Metrics의 정답 집합



범위 밖 질문(답이 없는 질문)은 의도적으로 포함한다. "모른다"가 정답인 문항으로 환각 내성을 확인한다.



합성 QA 생성 코드

합성 생성

```
class QAPair(BaseModel):
    question: str = Field(description=
        "문서를 안 본 사용자의 자연스러운 질문. "
        "문서 용어를 그대로 베끼지 말 것")
    answer: str = Field(description=
        "문서 조각의 내용만으로 작성한 정답")

GEN_SYS = (
    "다음 문서 조각만을 근거로, 사용자가 실제로 "
    "물을 법한 질문 1개와 그 정답을 작성한다.\n"
    "- 질문은 구어체로, 문서 문장을 베끼지 말 것\n"
    "- 문서 밖 지식을 절대 넣지 말 것")

qa = client.chat.completions.parse(
    model="gpt-5.4", temperature=0.7,
    messages=[...], response_format=QAPair)
```

- 생성에는 다양성이 필요하므로 temperature를 0.7 정도로 둔다. 값은 필요한 다양성에 따라 조정한다.
- "문서 표현을 그대로 옮기지 말 것"이라는 지시가 중요하다. 표현을 그대로 옮기면 검색이 지나치게 쉬워진다.
- "문서 밖 지식 금지"로 정답의 신뢰성을 확보한다.

생성 프롬프트의 제약이 평가셋 품질에 크게 작용한다.
생성 단계에서 걸러지지 않은 문항은 검증 단계에서 제외한다.



2단계 품질 검증 구조

비용이 낮은 검사를 먼저, 비용이 높은 검사를 나중에 한다. 필터 순서에 따라 검증 비용이 달라진다.

1단계 · 규칙 필터

n-gram 부분일치 표절 검사
비용 0 · 즉시 실행

검출 대상: 문서 문장을
그대로 옮긴 질문 (규칙 판별)



2단계 · LLM 검증자

answerable · natural · faithful
문항당 LLM 1회 호출

검출 대상: 답할 수 없는 질문
부자연스러운 말투, 정답 불일치



통과 문항

통과율 집계
유형 분포 점검

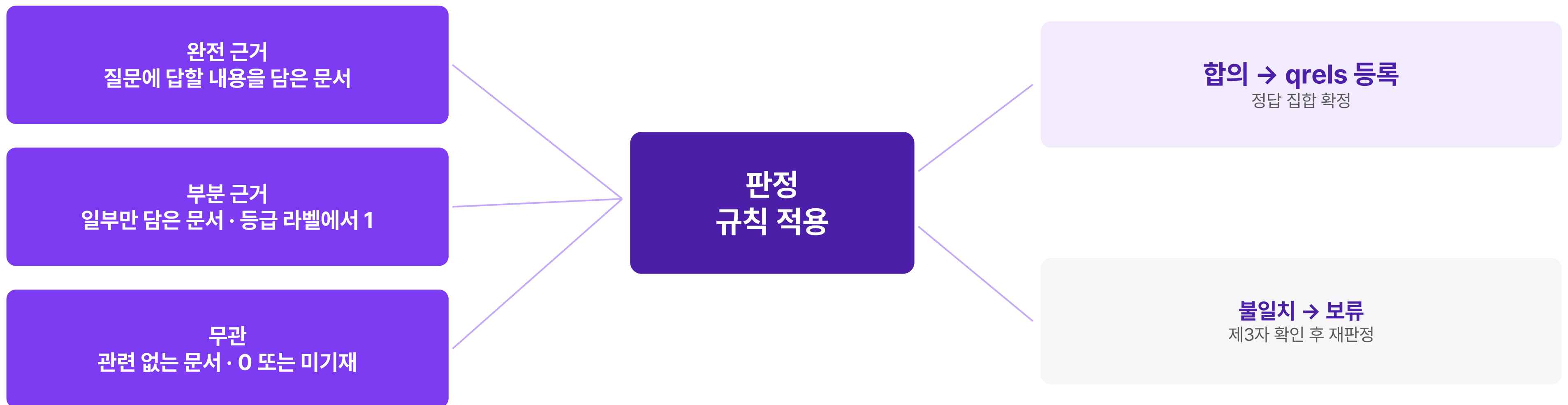
n 임계값 trade-off:
작으면 오탐 · 크면 미탐

범위 밖 질문(문서에 답이 없는 질문)은 의도적으로 포함한다. "모른다"가 정답인 문항으로 환각 내성을 확인한다.



근거 라벨링 규칙

라벨이 곧 정답 집합이 된다. 판정 기준을 먼저 문서로 남겨 두는 것을 권장한다.



라벨이 Recall의 상한이 된다. 관련 있는 문서에 라벨을 붙이지 않으면 그 문서를 찾아낸 검색기가 오히려 낮게 평가된다.



검수·저장·버전 관리

저장 스키마

```
# golden/v1.0/questions.jsonl
{
  "id": "q11",
  "question": "재구매 고객의 LTV는 신규 대비 몇 배인가",
  "answer": "신규구매 고객 대비 4.8배 높다",
  "evidence_ids": ["d03-p09"],
  "question_type": "factual",
  "context_type": "paragraph",
  "source_doc": "Dighty 이커머스 데이터 인사이트 보고서 2023",
  "reviewed_by": ["A", "B"],
  "corpus_version": "commerce-2026-09",
  "version": "v1.0"
}
```

- 두 사람이 각각 판정하고 불일치 문항만 모아 제3자가 확정하는 방식이 흔하다.
- 오염 검사는 세 항목을 본다. 문서 표현을 그대로 옮긴 질문, 질문에 정답이 드러난 문항, 특정 문서에 근거가 몰린 분포이다.
- 문항 파일과 코퍼스 버전을 하나의 태그로 묶는다. 태그가 다른 점수를 직접 비교하지 않는다.
- 평가 프레임워크와 관측 도구에 데이터셋으로 등록해 두면 실행 기록과 문항을 연결할 수 있다.

평가셋을 수정하면 기준선을 다시 측정한다. 문항이 바뀐 채로 이전 점수와 비교하면 개선인지 판단할 수 없다.

증강 구성: 개수·순서·단위



컨텍스트 구성의 세 변수

검색 결과가 같아도 개수·순서·단위를 어떻게 정하는지에 따라 답변과 비용이 달라진다.

1 문서 개수

컨텍스트에 넣는 Top-K를 늘리면
근거 회수는 늘어나고
무관한 문서도 함께 들어올 수 있다.

영향 범위

품질·비용

확인 방법

K를 바꿔 가며 답변 품질과
토큰 사용량을 함께 본다.

2 문서 순서

같은 문서라도 배치 순서에 따라
답변에서 참조되는 정도가
달라질 수 있다.

영향 범위

품질

확인 방법

순서만 바꾼 두 구성을
같은 질문으로 비교한다.

3 제공 단위

검색은 작은 청크로 하고
제공은 상위 문서로 넓히는
구성도 가능하다.

영향 범위

근거 범위

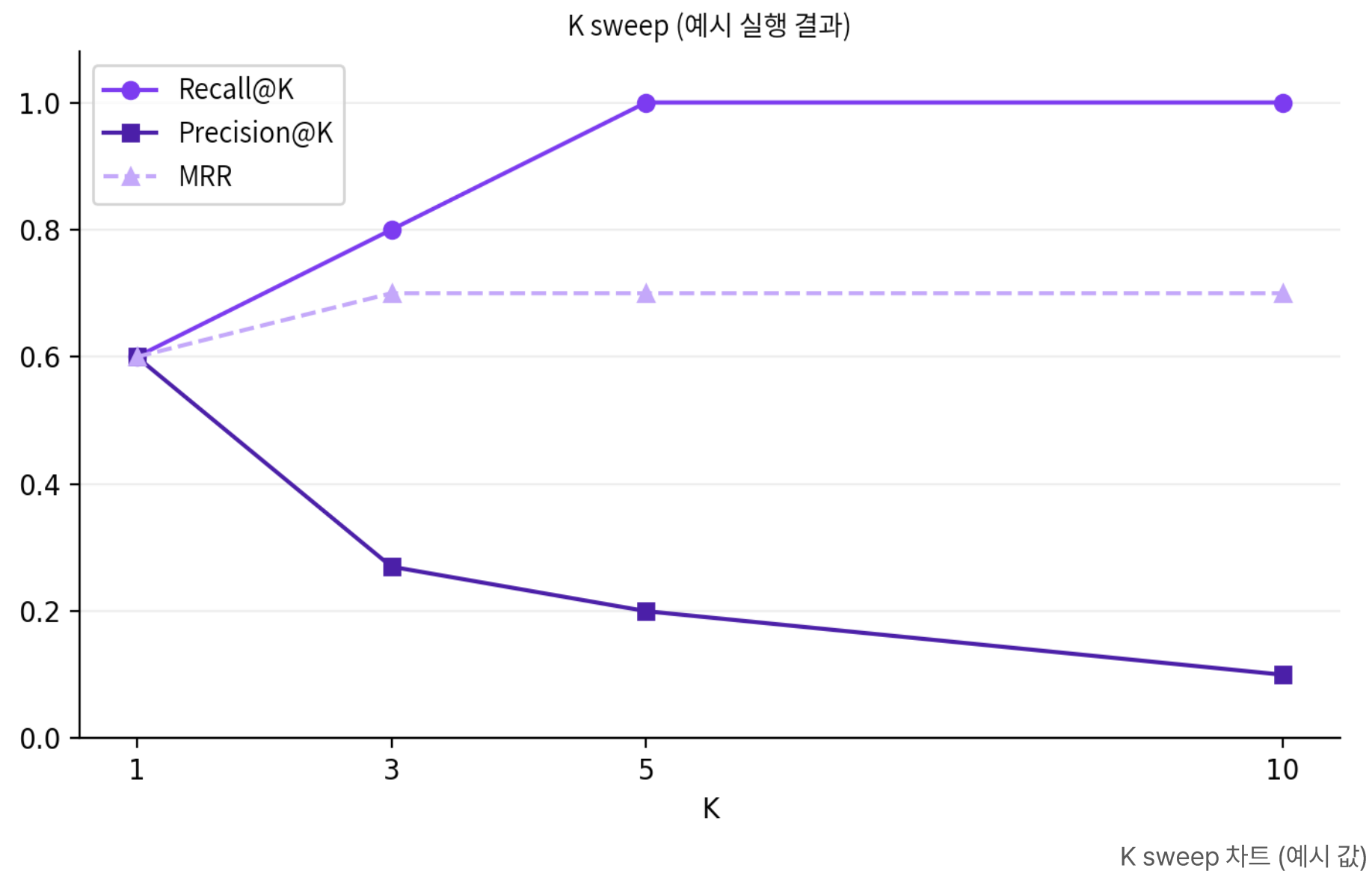
다루는 범위

개념만 정리하고, 실습에서는
개수와 순서에 집중한다.

세 변수는 검색 지표를 바꾸지 않는다. 같은 검색 결과에서 답변과 비용이 달라지는 구간이다.



K sweep과 trade-off



K를 키우면 Recall은 상승하고 Precision은 하락한다.

같은 평가셋에서 K를 1, 3, 5, 10으로 바꿔 가며 지표를 측정한 결과이다.

Recall은 K가 커질 때 줄어들지 않는다. 후보를 늘리면 회수는 늘어난다.
대신 Precision이 떨어지고, 컨텍스트가 길어져 토큰 비용·지연이 늘어난다.
K는 지표가 아니라 요구사항을 보고 정한다. 품질·비용·지연의 절충점을 선택한다.

K는 지표만으로 정하기 어렵다. 품질·토큰·지연 세 항목의 상한을 함께 확인한 뒤 정하는 것을 권장한다.

확인 항목	확인 방법	K에 주는 상한
품질	K를 1·3·5·10으로 바꿔 Recall@K·Precision@K를 측정한다	Recall이 더 오르지 않는 지점
토큰 예산	$K \times \text{평균 청크 길이}$ 로 컨텍스트 토큰을 계산한다	프롬프트 예산에서 역산한 K
지연	K를 키우며 검색·생성 소요 시간을 측정한다	체감 한도를 넘기지 않는 K
종합	세 상한 가운데 가장 작은 값을 K로 두는 것을 권장한다	요구사항이 바뀌면 다시 정한다

품질만 보고 K를 키우면 토큰과 지연 측면의 부담이 커질 수 있다. 세 상한 가운데 가장 작은 값을 K로 두는 것을 권장한다.



컨텍스트 배치와 입력 비용

앞부분이 매 요청 동일하면 캐시를 재사용할 수 있고, 바뀌면 재사용이 끊긴다.

고정 prefix · 검색 결과를 뒤에 배치

- 시스템 프롬프트와 지시문이 매 요청 동일하다
- 동일한 앞부분은 캐시에서 재사용되어 입력 단가가 내려가는 경우가 많다
- 근거가 뒤에 있어 답변에서 덜 참조될 수 있다

효과 · 입력 비용 절감

가변 prefix · 검색 결과를 앞에 배치

- 요청마다 앞부분이 달라진다
- 캐시 재사용이 끊겨 입력 토큰이 매번 새로 계산된다
- 근거가 앞에 있어 답변에서 더 많이 참조된다

효과 · 품질 유리, 비용 증가

캐시 재사용이 끊기는 지점



품질과 비용이 반대로 움직이는 구간이다. 두 배치의 차이는 답변 품질 지표와 입력 토큰으로 확인한다.